
◆ CONVERT TEXT INTO NUMERICAL FEATURES

`X_vectorized = vectorizer.fit_transform(X)`

`X_vectorized` → variable

`=` → assignment operator

`vectorizer` → TF-IDF Vectorizer object

`fit_transform()` → **method**

`X` → all email messages

👉 Use: **learn important words from the dataset** and **convert text into numerical features**.

◆ WHAT IS X?

Earlier:

`X = df['Messages']`

Suppose:

X

Hello friend

Win money now

Free lottery offer

Meeting at 5 PM

So:

X

contains all email messages.

◆ WHAT IS vectorizer?

Earlier:

`vectorizer = TfidfVectorizer(stop_words='english')`

This created a TF-IDF object.

Think:

vectorizer

is a machine.

Text

↓

vectorizer

↓ Numbers

◆ WHAT IS fit_transform()?

This is a method of the TF-IDF Vectorizer class.

Notice:

```
vectorizer.fit_transform(X)
```

The method belongs to:

```
vectorizer
```

Just like:

```
name.upper()
```

where:

```
upper()
```

is a method of String.

Similarly:

```
fit_transform()
```

is a method of the TF-IDF Vectorizer.

◆ WHY THE NAME fit_transform()?

Actually it combines two jobs:

```
fit()
```

```
+
```

```
transform()
```

into one method.

◆ WHAT DOES fit() MEAN?

```
fit
```

means:

Study the data

Learn information from it

TF-IDF scans all messages.

Example:

Hello friend

Win money now

Free lottery offer

It learns:

friend
win
money
free
lottery
offer

These become part of its vocabulary.

Think:

fit()

↓

Build a dictionary of words

◆ WHAT DOES transform() MEAN?

After learning the vocabulary:

transform()

means:

Convert text into numbers

Example:

Free money now

becomes something like:

[0.82, 0.71, 0.33]

The actual numbers depend on TF-IDF calculations.

◆ WHY USE fit_transform()?

Instead of writing:

```
vectorizer.fit(X)
X_vectorized = vectorizer.transform(X)
```

Scikit-Learn provides:

```
X_vectorized = vectorizer.fit_transform(X)
```

which does **both steps together**.

◆ WHAT DOES IT RETURN?

Many beginners think:

```
vectorizer.fit_transform(X)
```

returns a single number.

It doesn't.

It returns a matrix.

Example:

Message 1 → [0.1, 0.5, 0.0]

Message 2 → [0.8, 0.0, 0.4]

Message 3 → [0.0, 0.7, 0.9]

So:

`X_vectorized`

contains numerical representations of all messages.

◆ WHY STORE IT IN `X_vectorized`?

Because ML models cannot train directly on text.

Bad:

"Win money now"

Good:

[0.82, 0.71, 0.33]

Therefore:

`X_vectorized = vectorizer.fit_transform(X)`

stores the converted numerical data.

◆ COMPLETE FLOW

Messages

↓

`X`

↓

`vectorizer.fit_transform(X)`

↓

Learn vocabulary (fit)

↓

Convert text to numbers (transform)

↓

`X_vectorized`

↓

Used for model training

◆ SUMMARY

`X` → all email messages

`vectorizer` → TF-IDF object

`fit()` → learn vocabulary from messages

`transform()` → convert messages into numerical features

`fit_transform()` → perform both operations together

`X_vectorized` → numerical representation of all messages

👉 Use: prepare text data in a format that the ML model can understand and train on.

The most important thing to remember:

`fit()` = Learn words

`transform()` = Convert words to numbers

`fit_transform()` = Learn words and convert them to numbers in one step

During training

Suppose:

Message 1: Free money now

Message 2: Hello friend

Message 3: Win lottery

After:

`X_vectorized = vectorizer.fit_transform(X)`

you might get something like:

```
[  
[0.82, 0.71, 0.33],  
[0.00, 0.25, 0.00],  
[0.90, 0.10, 0.80]  
]
```

One row per message.

This is used to train:

`model.fit(X_train, y_train)`

Nothing is sent to Spring here.

During prediction (actual project running)

User enters:

Free money now

Spring sends:

```
{  
"emailText":"Free money now"  
}
```

to Flask.

Then Flask does:

```
vector = vectorizer.transform([email_text])
```

Notice:

```
[email_text]
```

contains only ONE email.

So the result looks like:

```
[  
[0.82, 0.71, 0.33]  
]
```

One row only.

Then:

```
prediction = model.predict(vector)
```

might return:

```
[1]
```

Then:

```
prediction[0]
```

gives:

```
1
```

Then Flask returns:

```
{  
"prediction":"SPAM"  
}
```

to Spring.

Does Spring receive the vector?

No. ❌

Spring never receives:

```
[0.82, 0.71, 0.33]
```

Spring only receives:

```
{  
"prediction":"SPAM"
```

```
}  
or  
{  
  "prediction": "HAM"  
}
```

Full flow

Spring

↓

"Free money now"

↓

Flask

↓

vectorizer.transform()

↓

[0.82, 0.71, 0.33]

↓

MultinomialNB

↓

[1]

↓

SPAM

↓

JSON

{

"prediction": "SPAM"

}

↓

Spring

So yes, the vector contains many numbers, but that's completely fine because it's used internally by the ML model. Spring never sees those numbers. It only sees the final prediction result. ✓